

Dashboard et suivi des performances BtoB

Toys & Models

Sommaire

01 Méthodologie

02 Dashboards

03 Recommandation

04 Annexes



Cahier des charges

Objectif : Création d'un tableau de bord dynamique et automatisé permettant à la Direction de suivre les indicateurs clés de performance de l'entreprise (ventes, finances, logistique, RH)



Périmètre du projet

- Analyse de la base de données existante
- Organisation des données autour de 4 axes principaux :
  - 💰 Ventes
  - 📊 Finances
  - 🚚 Logistique
  - 👥 Ressources humaines
- Conception d'un tableau de bord clair, interactif et synthétique



Contraintes & Attentes

- Respect des priorités définies par le client :
  - 🔴 Indicateurs obligatoires
  - 🟢 Indicateurs complémentaires
  - ⚫ Indicateurs optionnels
- Proposer des KPI supplémentaires en cas de limite technique ou pour enrichir l'analyse
- Assurer une lecture intuitive et une actualisation quotidienne

01

Méthodologie



Pourquoi un dashboard PowerBI ?

You can delete this slide when you're done editing the presentation



8

Bases de données

- Différentes informations difficiles à analyser
- Une vision globale impossible à dégager

MySQL

Power BI

KPIs

- Ressources humaines
- Ventes
- Finances
- Logistique

Notre méthode



COMPRENDRE

- Découverte de vos bases de données
- Prototypage du Modèle Conceptuel de données et du Modèle Logique de données



ANALYSER

- Analyse des KPIs demandés
- Création des tables de faits et de dimension pour la création de vos dashboards



VISUALISER

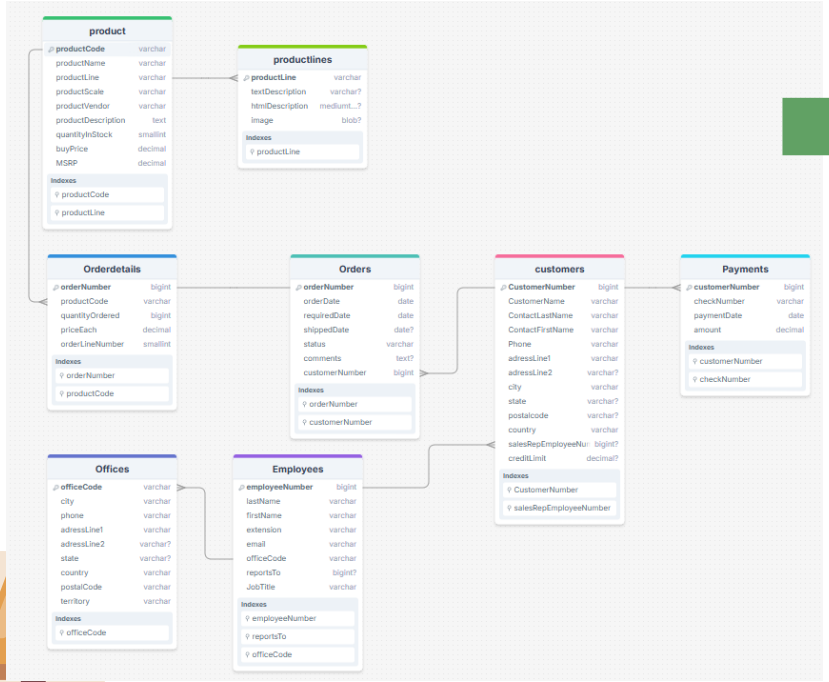
- Création de votre dashboard avec les KPIs essentiels pour vous aider à piloter votre entreprise.

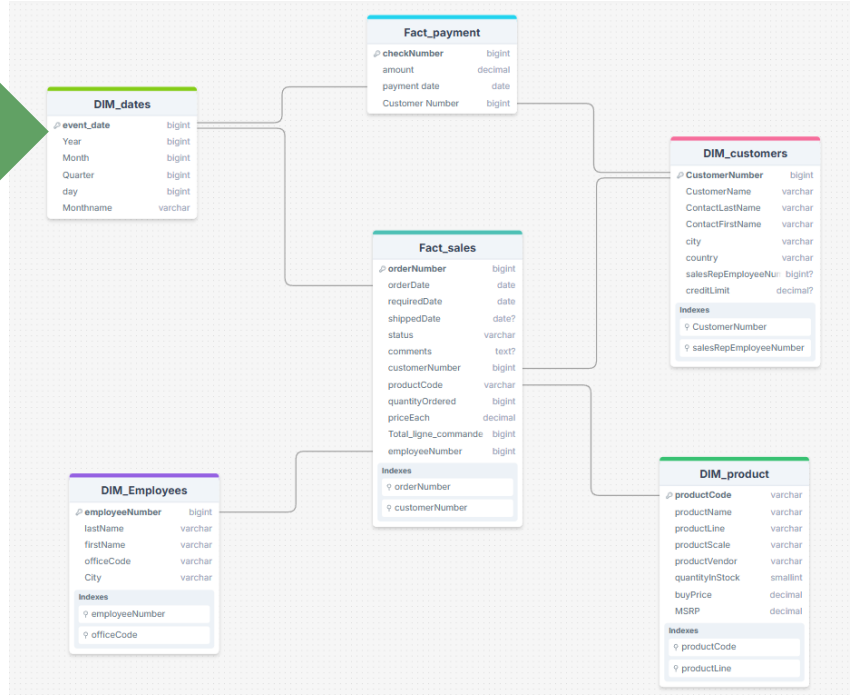




Préparation des vues pour PowerBI

PowerBI fonctionnant mieux avec un graphique dit en étoile, nous avons créé des vues permettant de créer des tables de faits et de dimensions :





02

Dashboards



ACCUEIL

- KPI ESSENTIELS
- C.A. MARGE
- C.A. CLIENT
- PERFORMANCES EMPLOYES
- PERFORMANCES BUREAUX
- GESTION DES PAIEMENTS
- FIDELITE CLIENT
- COMMANDES
- STOCKS

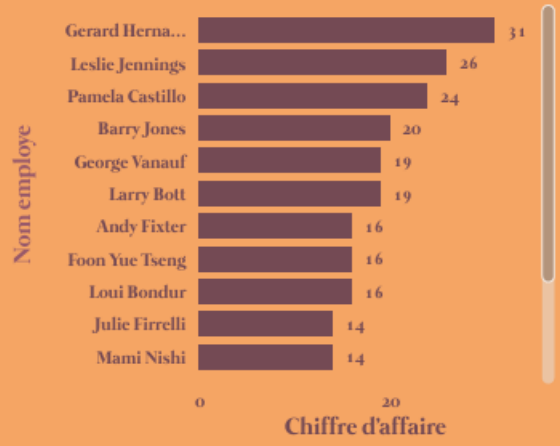


KPI ESSENTIELS

TOP 10 Clients (par CA Commande)

Nom clients	CA commande	Commercial
Euro+ Shopping Channel	$566 920	Gerard Hernandez
Mini Gifts Distributors Ltd.	$492 684	Leslie Jennings
Australian Collectors, Co.	$180 585	Andy Fixter
Muscle Machine Inc	$177 914	Foon Yue Tseng
Dragon Souveniers, Ltd.	$152 735	Mami Nishi
AV Stores, Co.	$148 410	Larry Bott
Saveley & Henriot, Co.	$130 305	Loui Bondur
Rovelli Gifts	$127 530	Pamela Castillo
Corporate Gift Ideas Co.	$118 185	Leslie Jennings

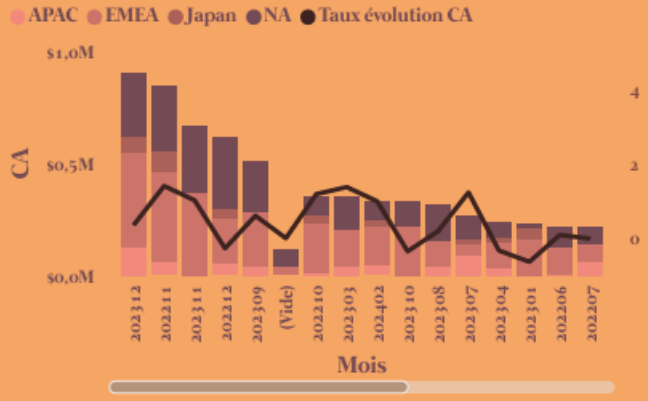
Top / Flop CA par employé



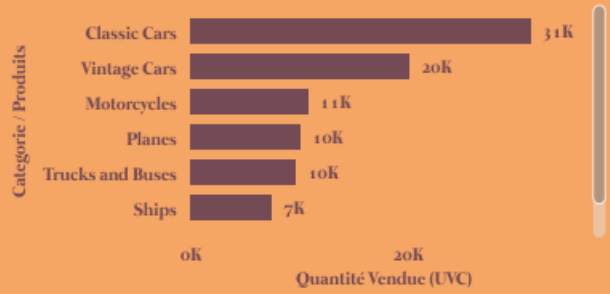
Stock des produits sous seuil critique



Evolution du CA mensuel par Région



Top / Flop (Ventes unités)



ACCUEIL

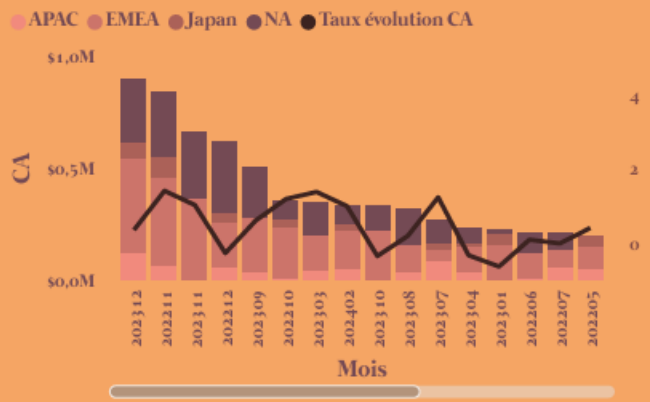
CA MARGE

Période souhaitée

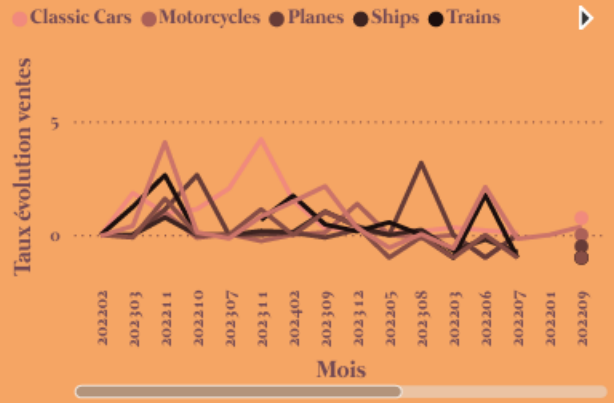
01/01/2022 — 28/02/2024

ACCUEIL

Evolution du CA mensuel par Région

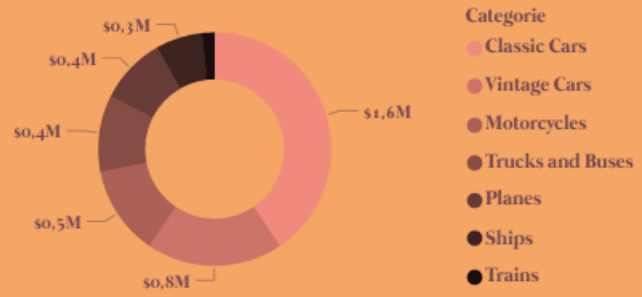


Ventes mensuelles par catégorie (évolution %)

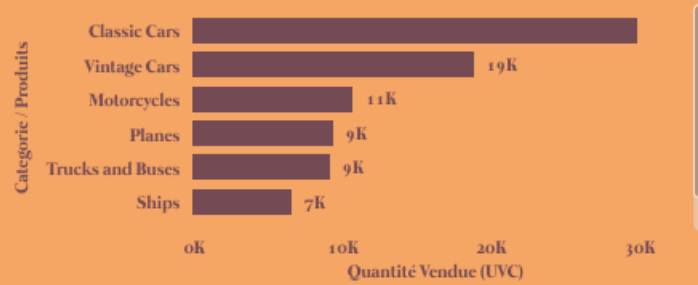


Chiffre d'Affaire	Marge Brute
$8,0M	$4,1M

Marge Brute par Categorie / Produits



Top / Flop (Ventes unités)

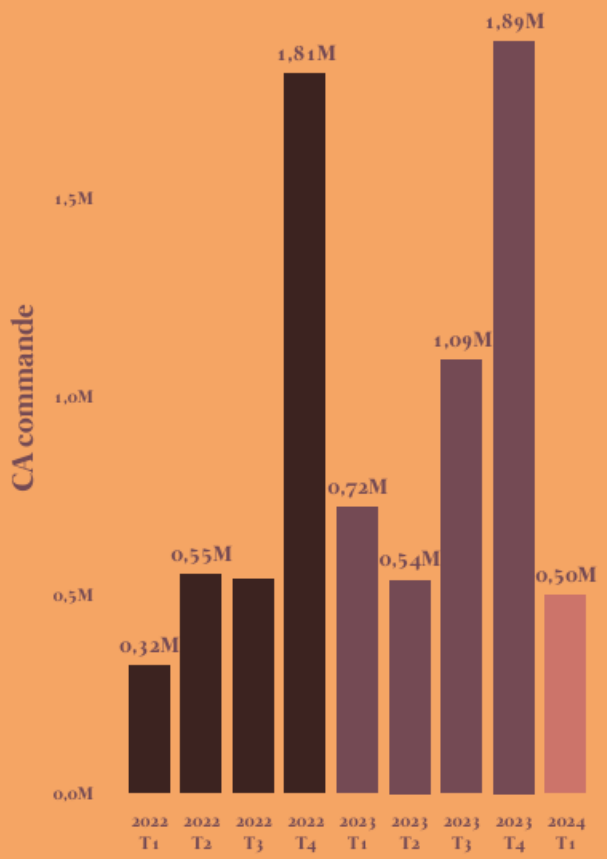


CA CLIENTS

Période souhaitée

25/01/2022 — 05/03/2024

Total CA commandé par trimestre



TOP 10 Clients (par CA Commande)

Nom clients	CA commande	Commercial
Euro+ Shopping Channel	$566 920	Gerard Hernandez
Mini Gifts Distributors Ltd.	$492 684	Leslie Jennings
Australian Collectors, Co.	$180 585	Andy Fixter
Muscle Machine Inc	$177 914	Foon Yue Tseng
Dragon Souveniers, Ltd.	$152 735	Mami Nishi
AV Stores, Co.	$148 410	Larry Bott
Saveley & Henriot, Co.	$130 305	Loui Bondur
Rovelli Gifts	$127 530	Pamela Castillo
Corporate Gift Ideas Co.	$118 185	Leslie Jennings
La Rochelle Gifts	$116 950	Gerard Hernandez

FLOP 10 Clients (par CA Commande)

Nom clients	CA commande	Commercial
Boards & Toys Co.	$7 918,6	Leslie Thompson
Mini Auto Werke	$15 131,0	Pamela Castillo
GiftsForHim.com	$20 220,0	Peter Marsh
Auto-Moto Classics Inc.	$21 554,3	Steve Patterson
Atelier graphique	$22 314,4	Gerard Hernandez
UK Collectables, Ltd.	$23 908,2	Larry Bott
Frau da Collezione	$25 358,3	Pamela Castillo
Royale Belge	$29 217,2	Pamela Castillo
Microscale Inc.	$29 230,4	Foon Yue Tseng
Double Decker Gift Stores, Ltd	$29 586,2	Larry Bott

ACCUEIL

KPI ESSENTIELS

PERFORMANCES EMPLOYES

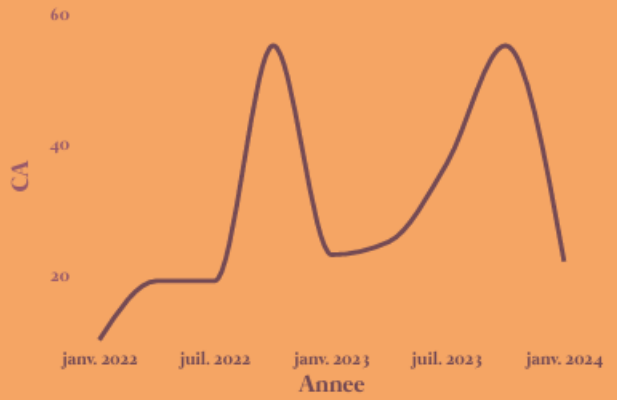
Selection de l'employé

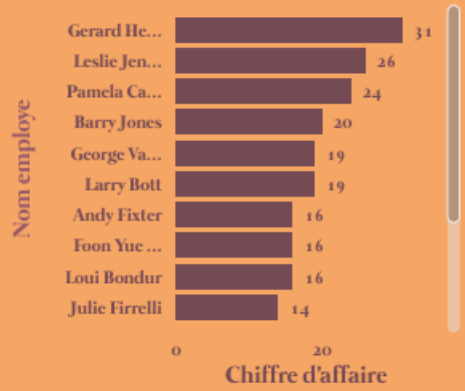
Tout

Periode souhaitee

25/01/2022 — 05/03/2024

Nom de l'employé	Chiffre d'affaire	Bureaux
Gerard Hernandez	962 660,38	Paris
Leslie Jennings	884 247,13	San Francisco
Pamela Castillo	741 394,75	Paris
Larry Bott	694 837,85	London
Barry Jones	676 887,37	London
George Vanauf	597 351,23	NYC
Loui Bondur	492 709,87	Paris
Andy Fixter	479 537,30	Sydney
Foon Yue Tseng	459 142,29	NYC
Mami Nishi	452 078,58	Tokyo





Paiements

7,95M

Ecart

445,93K

Ratio

14,39

ACCUEIL

PERFORMANCES BUREAUX

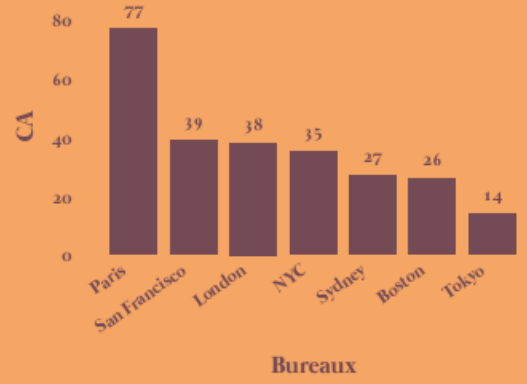
Bureaux	Pays	Chiffre d'affaires
Paris	France	2 584 242,47
London	UK	1 371 725,22
San Francisco	USA	1 231 780,16
NYC	USA	1 056 493,52
Sydney	Australia	896 461,22
Boston	USA	805 588,56
Tokyo	Japan	452 978,58

Période souhaitée

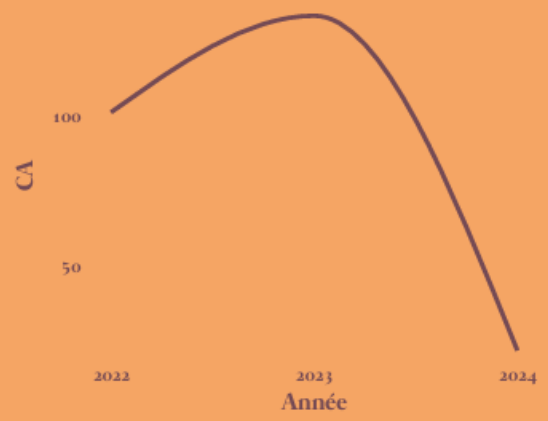
25/01/2022 — 05/03/2024

Bureaux	ratio de rentabilité (%)	taux de marge (%)
Boston	65,20	39,47
London	65,30	39,51
NYC	67,48	40,29
Paris	67,25	40,21
San Francisco	67,51	40,30
Sydney	65,59	39,61

Top / Flop CA par bureaux



Evolution du CA au fil des ans



Bénéfices brut

3,36M

ACCUEIL

GESTION PAIEMENTS

Débit

$119 812,96

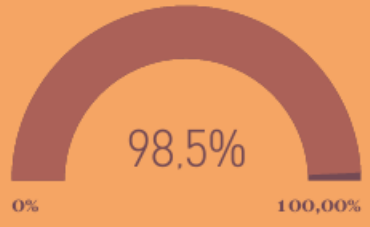
Paiement moyen

$31 941

Délai avant paiement

16

Taux de recouvrement



Période souhaitée

25/01/2022 — 05/03/2024

Taux de recouvrement par clients

Nom client	CA commande	Total_paye	Débit	Taux de recouvrement	Commercial
Boards & Toys Co.	$7 918,60	$4 465,85	$3 452,75	56 %	Leslie Thompson
Australian Gift Network, Co	$55 190,16	$33 757,85	$21 432,31	61 %	Andy Fixter
Volvo Model Replicas, Co	$66 694,82	$43 680,65	$23 014,17	65 %	Barry Jones
Corporate Gift Ideas Co.	$118 185,21	$85 559,12	$32 626,09	72 %	Leslie

Clients au-dessus du délai moyen de paiement

N° commande	Nom client	Date commande	Délai	Commercial
10272	Diecast Classics Inc.	05/08/2023	49	Steve Patterson
10250	The Sharp Gifts Warehouse	27/05/2023	41	Leslie Jennings
10222	Collectable Mini Designs Co.	06/03/2023	25	Leslie Thompson
10104	Euro+ Shopping Channel	15/02/2022	25	Gerard Hernandez
10129	Stylish Desk Decors	27/06/2022	25	Larry Bott
10159	Corporate Gift Ideas Co.	25/10/2022	24	Leslie Jennings
10217	Handji Gifts& Co	19/02/2023	24	Peter Marsh

Clients sous la moyenne paiement

customerName	Paiement moyen	Commercial
Boards & Toys Co.	$4 465,85	Leslie Thompson
Auto-Moto Classics Inc.	$7 184,75	Steve Patterson
Royale Belge	$7 304,30	Pamela Castillo
Atelier graphique	$7 438,12	Gerard Hernandez
Frau da Collezione	$12 679,16	Pamela Castillo
Microscale Inc.	$14 615,22	Foon Yue Tseng
Double Decker Gift Stores, Ltd	$14 793,08	Larry Bott
Cambridge Collectables Co.	$16 099,35	Julie Firrelli
Gift Ideas Corp.	$16 655,93	George Vanauf
Gifts4AllAges.com	$16 766,74	Steve Patterson

ACCUEIL

COMMANDES

Sélection du produit

Tout

Nombre de commandes

10K

Commandes Supérieures à la moyenne

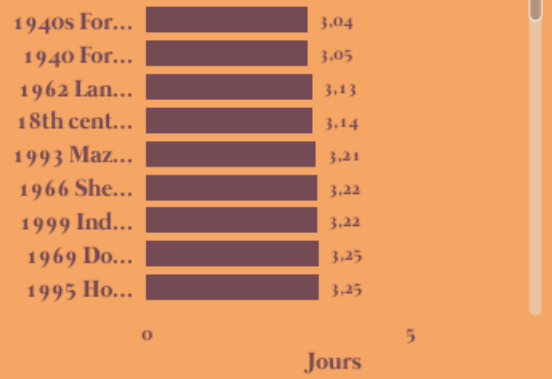


Période Souhaitée

28/01/2022

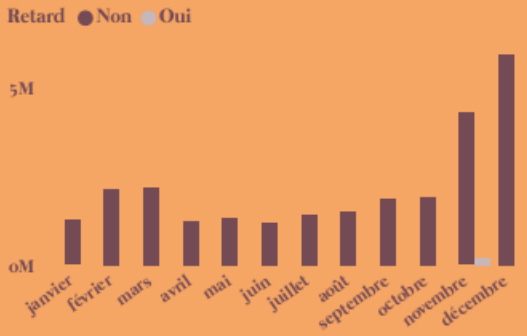
12/03/2024

Moyenne de traitement commande



Taux de commandes livrées en retard

par mois



ACCUEIL

STOCKS

Choix du seuil

1 100

Produits sous seuil

4

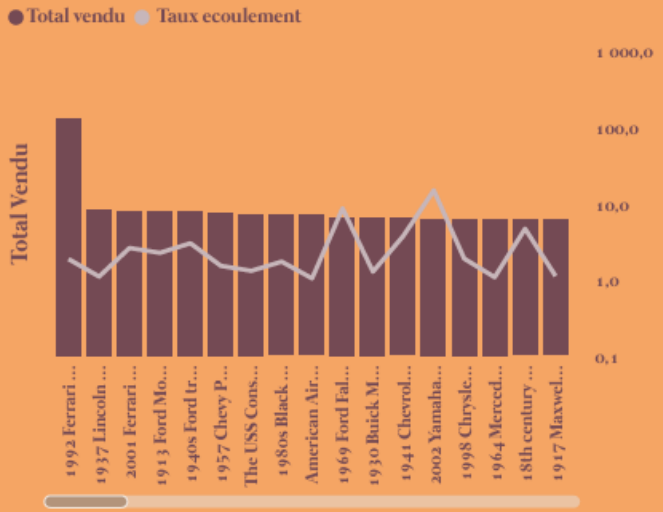
Période Souhaitée

28/01/2022 12/03/2024

Stock des produits sous seuil critique



Taux ecoulement des stocks



ACCUEIL

03

Recommandations



Confronter vos objectifs



Aucun objectif chiffré ou budget prévisionnel global dans le cahier des charges



Absence de vision sur :

- les tendances d’évolution (réalisé vs prévu)
- les écarts budgétaires



Mesure de la performance réelle et de l’excellence opérationnelle difficile

Pour aller plus loin

Ventes 

- Identifier les 20/80 et les poids morts
- Ecouler stock des produits à faible rentabilité via des mises en avant ou promo
- Inciter les commerciaux à pousser les produits à marge élevée dans les commandes

RH 

- Mettre en place un système de CA mensuel, trimestriel et annuel
- Sensibiliser au plus tôt et préparer les fêtes de fin d'année
- Viser un ratio de commandes/paiement unique

Logistiques 

- Déclencher le réapprovisionnement pour les produits sous le seuil critique
- Résoudre goulot d'étranglement pour diminuer la moyenne de traitement
- Renforcer les capacités logistiques pour la période de fin d'année

Finances 

- Créer une liaison entre les tables orders et payments en poussant la clé orderNumber dans Payments
- Repérer des opportunités d'étendre le marché (Allemagne)
- Adopter un CRM pour un meilleur suivi client

04

Annexes



Comparatif outils

Solution	Avantages	Inconvénients	Coûts de base	Contraintes techniques
Power BI	• Interface plutôt intuitive, drag-&-drop • Très bonne intégration avec l'écosystème Microsoft (Excel, Azure, SQL Server) • Large connectivité vers de nombreuses sources de données	• Performances parfois faibles avec de très grands volumes de données ou modèles complexes • L'apprentissage de fonctionnalités avancées (modélisation, DAX) peut demander un effort. • Écosystème « Microsoft-centré » : moins fluide dans des contextes mixtes avec beaucoup de technologies non Microsoft • Modèle de licences/partage peut devenir complexe et coûteux à monter en échelle	• Version gratuite disponible mais ne permet pas de travail collaboratif et ne s'exporte qu'en PDF. Pas de système d'alerte • 14 $/mois et utilisateur pour la version pro • 24$/mois pour la version premium (features AI par exemple)	• Desktop surtout Windows (macOS moins bien supporté) • Limites de taille des datasets selon licence (ex : Pro) • Pour certaines sources ou architecture non Microsoft, des connecteurs ou travaux supplémentaires peuvent être requis. • Besoin de maîtriser DAX/M et de bonnes pratiques de modélisation pour exploiter pleinement.
Tableau	• Très bon pour la visualisation avancée, l'exploration interactive des données • Meilleure gestion (souvent) des gros volumes ou architectures « data warehouse/enterprise ». • Support multiplateforme (Windows/macOS) et large communauté.	• Coût global souvent élevé, surtout à l'échelle. • Courbe d'apprentissage plus prononcée pour les fonctions avancées (calculs, blending, etc.). • Moins d'intégration native «clé-en-main» avec certains écosystèmes Microsoft/Excel que Power BI. • Export de dashboards vers PDF/PowerPoint parfois peu optimal.	Exemple de tarification : • US$ 75/utilisateur/mois (facturation annuelle) pour l'édition «Standard». • Explorer et Viewer à des tarifs plus bas (ex : ~US$ 42 et ~US$ 15/mois selon le rôle) • Pour usage large entreprise, coûts d'infrastructure, licences, gouvernance peuvent vite monter	• Bien que supportant de gros volumes, des efforts d'optimisation et de gouvernance sont nécessaires pour maintenir de bonnes performances • Pour des déploiements on-premise, gestion de l'infrastructure + licences plus lourdes. • Création et gestion de sources de données + nettoyage/ETL souvent en amont. • Pour certains projets, personnalisation / visualisations très pointues peuvent nécessiter des extensions ou codage.
Looker Studio	• Gratuit pour la version de base (ce qui le rend très attractif pour petites structures, freelances, début) • Interface simple, «drag & drop», assez accessible pour des utilisateurs non-techniques • Bonne intégration avec l'écosystème Google (Sheets, BigQuery, Google Analytics, Ads)	• Moins adapté aux analyses très avancées, aux transformations complexes ou aux très grands volumes de données. • Connecteurs «non-Google» souvent payants ou moins performants. • Performance / rapidité parfois critiquées (latence, volume, refresh) • Pour un usage entreprise important, la version gratuite peut montrer ses limites en gouvernance, sécurité ou fonctionnalités	Tarification à négocier, ordres de grandeur : • Version de base gratuite. • Version «Pro» annoncée autour de US$ 9/utilisateur/mois dans certains cas. • Pour usage «Enterprise» très large, certains montants (US$ 35 000-60 000/an) ont été évoqués.	• Largement orienté cloud, dépend d'une connexion internet. • Pour charger et manipuler de gros volumes ou faire beaucoup de calculs, il faudra externaliser la logique (ex : BigQuery). • Certaines limitations (nombre de widgets, volumétrie) dans la version gratuite. • Gouvernance, sécurité, versioning moins mature que certaines solutions «pure BI enterprise».

Requêtes SQL RH

```
CREATE VIEW perfs_employes AS
        SELECT employeeNumber, lastName, firstName, sum(orderdetails.quantityOrdered * orderdetails.priceEach) AS
chiffre_d_affaire
        FROM employees
        INNER JOIN customers
        ON employees.employeeNumber = customers.salesRepEmployeeNumber
        INNER JOIN orders
        ON customers.customerNumber = orders.customerNumber
        INNER JOIN orderdetails
        ON orders.orderNumber = orderdetails.orderNumber
        GROUP BY employeeNumber
        ORDER BY chiffre_d_affaire DESC
;

CREATE VIEW perfs_bureaux AS
        SELECT offices.country, offices.city, offices.officeCode, sum(orderdetails.quantityOrdered * orderdetails.priceEach) AS
chiffre_d_affaire
        FROM offices
        INNER JOIN employees
        ON offices.officeCode = employees.officeCode
        INNER JOIN customers
        ON employees.employeeNumber = customers.salesRepEmployeeNumber
        INNER JOIN orders
        ON customers.customerNumber = orders.customerNumber
        INNER JOIN orderdetails
        ON orders.orderNumber = orderdetails.orderNumber
        GROUP BY officeCode
        ORDER BY officeCode
;
```

Requêtes SQL RH



```
CREATE VIEW ratio_commandes_paiements AS
        SELECT chiffre_d_affaire, total_payments, commandes.chiffre_d_affaire - paiements.total_payments AS ecart,
                IF (commandes.chiffre_d_affaire = 0, NULL, paiements.total_payments / commandes.chiffre_d_affaire) AS rat
        FROM (
                SELECT employeeNumber, lastName, firstName, sum(orderdetails.quantityOrdered * orderdetails.priceEach) AS
chiffre_d_affaire
                FROM employees
                INNER JOIN customers
                ON employees.employeeNumber = customers.salesRepEmployeeNumber
                INNER JOIN orders
                ON customers.customerNumber = orders.customerNumber
                INNER JOIN orderdetails
                ON orders.orderNumber = orderdetails.orderNumber
                GROUP BY employeeNumber
                ) AS commandes
        INNER JOIN (
                SELECT firstname, lastname, employeeNumber, sum(payments.amount) AS total_payments
                FROM employees
                INNER JOIN customers
                ON employees.employeeNumber = customers.salesRepEmployeeNumber
                INNER JOIN payments
                ON customers.customerNumber = payments.customerNumber
                GROUP BY employeeNumber
                ) AS paiements
        ON commandes.employeeNumber = paiements.employeeNumber
;
```



Requêtes SQL RH



```
CREATE VIEW employe_par_region AS
        SELECT employeeNumber, lastName, firstName, country, jobTitle
        FROM employees
        INNER JOIN offices
        ON employees.officeCode = offices.officeCode
;

CREATE VIEW plus_grosse_commande_par_employe AS
        SELECT c.employeeNumber, c.customerName, c.orderNumber, c.total_commande AS montant_commande
        FROM (                  /* On calcule d'abord le montant des commandes par employé*/
                SELECT employees.employeeNumber, customers.customerName, orders.orderNumber, SUM(orderdetails.quantityOrdered * orderdetails.priceEach) AS total_commande
                FROM employees
                INNER JOIN customers
                ON employees.employeeNumber = customers.salesRepEmployeeNumber
                INNER JOIN orders
                ON customers.customerNumber = orders.customerNumber
                INNER JOIN orderdetails
                ON orders.orderNumber = orderdetails.orderNumber
                GROUP BY employees.employeeNumber, customers.customerName, orders.orderNumber
        ) AS c

        INNER JOIN (  /* Ici on détermine la commande qui a généré le plus gros chiffre d'affaire */
                SELECT employeeNumber, MAX(total_commande) AS max_commande
                FROM (
                                SELECT employees.employeeNumber, orders.orderNumber, SUM(orderdetails.quantityOrdered * orderdetails.priceEach) AS total_commande
                                FROM employees
                                INNER JOIN customers
                                ON employees.employeeNumber = customers.salesRepEmployeeNumber
                                INNER JOIN orders
                                ON customers.customerNumber = orders.customerNumber
                                INNER JOIN orderdetails
                                ON orders.orderNumber = orderdetails.orderNumber
                                GROUP BY employees.employeeNumber, orders.orderNumber
                        ) AS totals
                        GROUP BY employeeNumber
                ) AS m

        ON c.employeeNumber = m.employeeNumber                  /* Ici on joint le montant des commandes et la plus grosse commande d'un même employé*/
        AND c.total_commande = m.max_commande                   /* Ici on filtre les commandes d'un même employé et on ne garde que la celle la plus élevée*/
                                                                        /* Ces conditions sont là pour dans le cas où un employé aurait deux
commandes avec le même montant max*/
        ORDER BY montant_commande DESC
.
```



Requêtes SQL RH



```
CREATE VIEW plus_grosse_commande_par_client AS
	SELECT customerName, MAX(total_commande) AS max_commande
		FROM (
				SELECT customers.customerName, orders.orderNumber, SUM(orderdetails.quantityOrdered * orderdetails.priceEach) AS total_commande
				FROM employees
				INNER JOIN customers
				ON employees.employeeNumber = customers.salesRepEmployeeNumber
				INNER JOIN orders
				ON customers.customerNumber = orders.customerNumber
				INNER JOIN orderdetails
				ON orders.orderNumber = orderdetails.orderNumber
				GROUP BY customers.customerName, orders.orderNumber
			) AS totals
			GROUP BY customerName
	ORDER BY max_commande DESC
;

CREATE VIEW perfs_par_date_employe AS
	SELECT employees.employeeNumber, employees.lastName, employees.firstName, orders.shippedDate, sum(orderdetails.quantityOrdered * orderdetails.priceEach) AS
chiffre_d_affaire
	FROM employees
	INNER JOIN customers
	ON employees.employeeNumber = customers.salesRepEmployeeNumber
	INNER JOIN orders
	ON customers.customerNumber = orders.customerNumber
	INNER JOIN orderdetails
	ON orders.orderNumber = orderdetails.orderNumber
	WHERE orders.status = "Shipped"
	GROUP BY employees.employeeNumber, orders.shippedDate
	ORDER BY employees.employeeNumber
;
```



Requêtes SQL RH



```
CREATE VIEW perfs_par_date_bureaux AS
        SELECT offices.country, offices.city, offices.officeCode, orders.shippedDate, sum(orderdetails.quantityOrdered * orderdetails.priceEach) AS chiffre_d_affaire
        FROM offices
        INNER JOIN employees
        ON offices.officeCode = employees.officeCode
        INNER JOIN customers
        ON employees.employeeNumber = customers.salesRepEmployeeNumber
        INNER JOIN orders
        ON customers.customerNumber = orders.customerNumber
        INNER JOIN orderdetails
        ON orders.orderNumber = orderdetails.orderNumber
        WHERE orders.status = "Shipped"
        GROUP BY officeCode, orders.shippedDate
        ORDER BY officeCode
;

CREATE VIEW rentabilite_bureaux AS
        WITH rentabilite AS (
                SELECT  offices.country, offices.city, offices.officeCode, products.buyPrice, orderdetails.priceEach, orderdetails.quantityOrdered,
                (orderdetails.priceEach - products.buyPrice) * orderdetails.quantityOrdered AS Rentability
                FROM offices
                INNER JOIN employees
                ON offices.officeCode = employees.officeCode
                INNER JOIN customers
                ON employees.employeeNumber = customers.salesRepEmployeeNumber
                INNER JOIN orders
                ON customers.customerNumber = orders.customerNumber
                INNER JOIN orderdetails
                ON orders.orderNumber = orderdetails.orderNumber
                INNER JOIN products
                ON products.productCode = orderdetails.productCode
                )
        SELECT rentabilite.country, rentabilite.city, rentabilite.officeCode, sum(rentabilite.buyPrice) AS prix_d_achat, sum(rentabilite.priceEach) AS prix_de_vente,
        sum(rentabilite.quantityOrdered) AS quantite_vendue, sum(rentabilite.rentability) AS benefices,
        ((sum(rentabilite.priceEach) - sum(rentabilite.buyPrice)) / sum(rentabilite.buyPrice)) * 100 AS ratio_rentabilite,
        ((sum(rentabilite.priceEach) - sum(rentabilite.buyPrice)) / sum(rentabilite.priceEach)) * 100 AS taux_de_marge
        FROM rentabilite
        GROUP BY rentabilite.city, rentabilite.officeCode
;
```

Requêtes SQL Vente



```
#SS Requete pour récupérer les CA mensuels par Région
WITH CA_mensuel AS(
        SELECT DATE_FORMAT(orders.orderDate, '%Y-%m') AS AnneeMois, SUM(orderdetails.quantityOrdered * orderdetails.priceEach) AS CA, offices.territory AS Region
        FROM orderdetails
        JOIN orders ON orderdetails.orderNumber = orders.orderNumber
        JOIN customers ON customers.customerNumber = orders.customerNumber
        JOIN employees ON employees. employeeNumber = customers.salesRepEmployeeNumber
        JOIN offices ON offices.officeCode = employees.officeCode
        GROUP BY offices.territory, DATE_FORMAT(orders.orderDate, '%Y-%m'))

#Bloc de l'evolution du CA par mois et par Région + Taux evolution avec LAG
SELECT
    AnneeMois,
    Region,
    CA,
    LAG(CA) OVER (PARTITION BY Region ORDER BY AnneeMois) AS CA_Precedent,
    (CA - LAG(CA) OVER (PARTITION BY Region ORDER BY AnneeMois))
     / LAG(CA) OVER (PARTITION BY Region ORDER BY AnneeMois) * 100 AS Taux_evolution
FROM CA_mensuel
ORDER BY Region, AnneeMois;


#Produits les plus/moins vendus par catégorie : Identifier les produits les plus performants dans chaque catégorie.

 SELECT products.productLine, products.productName, SUM(orderdetails.quantityOrdered)as QtéVendues
 FROM orderdetails
 JOIN products ON  products.productCode = orderdetails.productCode
 GROUP BY products.productLine, products.productName
 ORDER BY SUM(orderdetails.quantityOrdered) DESC;

 #Calcul de le marge brute par produit et categorie
SELECT productName as Produits, productLine as Categorie, ( MSRP - buyPrice)*quantityOrdered as MargeBrute
FROM products
JOIN orderdetails ON  products.productCode = orderdetails.productCode
ORDER BY (MSRP - buyPrice) DESC;
```

Requêtes SQL Vente



```
#Taux evolution mensuel des ventes par catégorie
        #Calcul des ventes mensuelles par categories
WITH Vente_categorie_mensuelle AS(
        SELECT  DATE_FORMAT(orderDate,'%Y-%m') AS AnneeMois,
    products.productLine as Categorie, SUM(orderdetails.quantityOrdered) as Ventes
        FROM orderdetails
        JOIN products ON products.productCode = orderdetails.productCode
    JOIN orders ON orders.orderNumber = orderdetails.orderNumber
        GROUP BY DATE_FORMAT(orderDate, '%Y-%m'),products.productLine)

SELECT AnneeMois, Categorie, Ventes,
        LAG(Ventes) OVER (PARTITION BY Categorie ORDER BY AnneeMois) AS Ventes_Precedentes,
    (Ventes - LAG(Ventes) OVER (PARTITION BY Categorie ORDER BY AnneeMois))
     / LAG(Ventes) OVER (PARTITION BY Categorie ORDER BY AnneeMois) * 100 AS Taux_evolution
FROM Vente_categorie_mensuelle
ORDER BY AnneeMois, Categorie;

 #Calcul du panier moyen
SELECT SUM(quantityOrdered * priceEach) / COUNT(DISTINCT orderNumber) as Panier_Moyen
FROM orderdetails;


# Calcul du taux de rachat
        #Clients_recurrents
WITH clientsrecurrent AS(
        SELECT customerNumber
        FROM orders
        GROUP BY customerNumber
        HAVING COUNT(orderNumber) >= 2),
        #Clients total
        clientstotal AS(
        SELECT COUNT(*)
        FROM customers
)
#Taux repeat Customer
SELECT
        (
        SELECT COUNT(*)
        FROM clientsrecurrent)
                                /
    (SELECT COUNT(*)
    FROM customers) *100 AS Tauxderachat ;
```

Requêtes SQL Vente



```
#Vue CA mensuel
#Vue CA mensuel
CREATE VIEW view_CA_mensuel AS
        SELECT DATE_FORMAT(orders.orderDate, '%Y-%m') AS AnneeMois, SUM(orderdetails.quantityOrdered * orderdetails.priceEach) AS CA, offices.territory AS Region
        FROM orderdetails
        JOIN orders ON orderdetails.orderNumber = orders.orderNumber
        JOIN customers ON customers.customerNumber = orders.customerNumber
        JOIN employees ON employees. employeeNumber = customers.salesRepEmployeeNumber
        JOIN offices ON offices.officeCode = employees.officeCode
        GROUP BY offices.territory, DATE_FORMAT(orders.orderDate, '%Y-%m');


#Vue l'evolution du CA par mois et par Région + Taux evolution
CREATE VIEW view_evol_CA AS
SELECT
    AnneeMois,
    Region,
    CA,
    LAG(CA) OVER (PARTITION BY Region ORDER BY AnneeMois) AS CA_Precedent,
    (CA - LAG(CA) OVER (PARTITION BY Region ORDER BY AnneeMois))
     / LAG(CA) OVER (PARTITION BY Region ORDER BY AnneeMois) * 100 AS Taux_evolution
FROM CA_mensuel
ORDER BY Region, AnneeMois;

#Vue Produits les plus/moins vendus par catégorie

 CREATE VIEW view_ventes_categorie AS
 SELECT products.productLine, products.productName, SUM(orderdetails.quantityOrdered)as QtéVendues
 FROM orderdetails
 JOIN products ON  products.productCode = orderdetails.productCode
 GROUP BY products.productLine, products.productName
 ORDER BY SUM(orderdetails.quantityOrdered) DESC;

  #Vue de la marge brut par produit et categorie
CREATE VIEW view_margebrute AS
SELECT productName as Produits, productLine as Categorie, ( MSRP - buyPrice)*quantityOrdered as MargeBrute
FROM products
JOIN orderdetails ON  products.productCode = orderdetails.productCode
ORDER BY (MSRP - buyPrice) DESC;
```

Requêtes SQL Vente

```
# Evolution mensuelle des ventes par catégorie
CREATE VIEW view_evol_vente AS
WITH Vente_categorie_mensuelle AS(
        SELECT  DATE_FORMAT(orderDate,'%Y-%m') AS AnneeMois,
    products.productLine as Categorie, SUM(orderdetails.quantityOrdered) as Ventes
        FROM orderdetails
        JOIN products ON products.productCode = orderdetails.productCode
    JOIN orders ON orders.orderNumber = orderdetails.orderNumber
        GROUP BY DATE_FORMAT(orderDate, '%Y-%m'),products.productLine)

SELECT AnneeMois, Categorie, Ventes,
        LAG(Ventes) OVER (PARTITION BY Categorie ORDER BY AnneeMois) AS Ventes_Precedentes,
    (Ventes - LAG(Ventes) OVER (PARTITION BY Categorie ORDER BY AnneeMois))
     / LAG(Ventes) OVER (PARTITION BY Categorie ORDER BY AnneeMois) * 100 AS Taux_evolution
FROM Vente_categorie_mensuelle
ORDER BY AnneeMois, Categorie;

#Vue Rachat
CREATE VIEW view_taux_rachat AS
WITH clientsrecurrent AS(
        SELECT customerNumber
        FROM orders
        GROUP BY customerNumber
        HAVING COUNT(orderNumber) >= 2),
        #Clients total
        clientstotal AS(
        SELECT COUNT(*)
        FROM customers
)
#Taux repeat Customer
SELECT
        (
        SELECT COUNT(*)
        FROM clientsrecurrent)
                                /
    (SELECT COUNT(*)
    FROM customers) *100 AS Tauxderachat;

#Vue clients recurrents
CREATE VIEW clientsrecurrent AS(
        SELECT customerNumber
        FROM orders
        GROUP BY customerNumber
        HAVING COUNT(orderNumber) >= 2)
```

Requêtes SQL Logistique



```sql
-- 1. Stock produits sous seuil critique :

CREATE OR REPLACE VIEW seuil_critique AS
SELECT
    productCode,
    productName,
    quantityInStock
FROM
    products
WHERE
    quantityInStock < 20;-- seuil critique a définir

SELECT *
FROM seuil_critique
ORDER BY quantityInStock ASC;

```



Requêtes SQL Logistique



```sql
-- 2. Durée des traitements de commande


  CREATE OR REPLACE VIEW traitement_commande AS
   SELECT
    products.productName,
    products.productCode,
    orders.orderNumber,
    orderDate,
    shippedDate,
    DATEDIFF(shippedDate, orderDate) AS traitement_commande  -- diffence entre la date d'expedition et la commande et la date
FROM
orders
join orderdetails on orders.orderNumber = orderdetails.orderNumber
join products on products.productCode = orderdetails.productCode
WHERE
    shippedDate IS NOT NULL;

SELECT *
FROM traitement_commande;
```

Requêtes SQL Logistique



```sql
-- Calculer duree moyenne de livraison :

 CREATE OR REPLACE VIEW delivery_duration AS
SELECT
products.productName,
 AVG(DATEDIFF(shippedDate, orderDate)) AS avg_delivery_duration -- moyenne de la difference entre la date d'expedition et la date attendu
FROM
  orders
  join orderdetails on orders.orderNumber = orderdetails.orderNumber
  join products on products.productCode = orderdetails.productCode
WHERE
  orders.shippedDate IS NOT NULL
  GROUP BY
 productName;


 SELECT *
FROM delivery_duration;
```



Requêtes SQL Logistique



```sql
-- 3. Taux d'écoulement des stocks
  -- stock actuel de chaque produits

CREATE OR REPLACE VIEW quantityinStock AS
SELECT
quantityInStock, productName, productCode
FROM products
ORDER BY
quantityinStock  DESC;

 SELECT *
FROM quantityinStock;
```

```sql
-- trouver quantité vendues pour chaque produits
CREATE OR REPLACE VIEW total_vendu AS
SELECT
products.productCode,
products.productName,
SUM(quantityOrdered) AS total_vendu -- somme quantité vendu alias total_vendu
FROM orderdetails
JOIN products on orderdetails.productCode = products.productCode -- jointure codedetails: (nbr commande) au productCode(details nom,etc..)
GROUP BY products.productCode
ORDER BY total_vendu DESC;

 SELECT *
FROM total_vendu;
```

Requêtes SQL Logistique



```sql
-- 3. Taux d'écoulement des stocks
  -- stock actuel de chaque produits

CREATE OR REPLACE VIEW quantityinStock AS
SELECT
quantityInStock, productName, productCode
FROM products
ORDER BY
quantityinStock  DESC;

 SELECT *
FROM quantityinStock;
```

```sql
-- trouver quantité vendues pour chaque produits
CREATE OR REPLACE VIEW total_vendu AS
SELECT
products.productCode,
products.productName,
SUM(quantityOrdered) AS total_vendu -- somme quantité vendu alias total_vendu
FROM orderdetails
JOIN products on orderdetails.productCode = products.productCode -- jointure codedetails: (nbr commande) au productCode(details nom,etc..)
GROUP BY products.productCode
ORDER BY total_vendu DESC;

 SELECT *
FROM total_vendu;
```

Requêtes SQL Logistique



```sql
CREATE OR REPLACE VIEW taux_ecoulement AS
SELECT
products.productCode,
quantityInStock,
products.productName,
SUM(quantityOrdered) / quantityInStock AS Taux_ecoulement -- C'est un ratio_indicatif , Taux d'écoulement = (quantité vendue) / (quantité en stock)
FROM orderdetails
JOIN products on orderdetails.productCode = products.productCode
GROUP BY products.productCode, quantityInStock, products.productName
ORDER BY Taux_ecoulement DESC;

SELECT *
FROM taux_ecoulement;
```



Requêtes SQL Logistique



```sql
CREATE OR REPLACE VIEW commandes_retard AS
SELECT
  productName,
  orders.orderNumber,
  requiredDate,
  shippedDate,
  DATEDIFF(shippedDate, requiredDate) AS Jours_de_retard, -- difference entre la date d'expedition et la date attendue
  CASE
    WHEN shippedDate > requiredDate THEN 'Oui'
    ELSE 'Non'
  END AS Retard
FROM orders
JOIN orderdetails on orderdetails.orderNumber = orders.orderNumber
JOIN products on products.productCode = orderdetails.productCode
WHERE shippedDate IS NOT NULL;

SELECT *
FROM commandes_retard
ORDER BY Jours_de_retard DESC;
```

Requêtes SQL Finances



```sql
CREATE OR REPLACE VIEW commandes_retard AS
SELECT
  productName,
  orders.orderNumber,
  requiredDate,
  shippedDate,
  DATEDIFF(shippedDate, requiredDate) AS Jours_de_retard, -- difference entre la date d'expedition et la date attendue
  CASE
    WHEN shippedDate > requiredDate THEN 'Oui'
    ELSE 'Non'
  END AS Retard
FROM orders
JOIN orderdetails on orderdetails.orderNumber = orders.orderNumber
JOIN products on products.productCode = orderdetails.productCode
WHERE shippedDate IS NOT NULL;

SELECT *
FROM commandes_retard
ORDER BY Jours_de_retard DESC;
```